

1 PHẦN 3: TRIỂN KHAI VÀ PHÂN TÍCH CHUYÊN SÂU THUẬT TOÁN

1.1 Phân tích Thuật toán Lựa chọn và Cơ sở Khoa học

1.1.1 Bản chất của Bài toán Sudoku dưới góc độ Khoa học Máy tính

Bài toán giải ô số Sudoku kích thước 9×9 bản chất là một bài toán thỏa mãn ràng buộc (*Constraint Satisfaction Problem - CSP*) và đã được chứng minh thuộc phân lớp bài toán **NP-đầy đủ (NP-complete)** khi mở rộng tổng quát hóa cấu trúc ma trận dưới dạng kích thước $N \times N$.

Một bàn cờ Sudoku tiêu chuẩn bao gồm một lưới ma trận vuông kích thước 9×9 , chia làm 9 vùng con (block/box) kích thước 3×3 . Tại một trạng thái cấu hình bất kỳ, hệ thống cần gán các giá trị nguyên $x_{ij} \in \{1, 2, \dots, 9\}$ vào các ô trống tại tọa độ (i, j) sao cho đồng thời thỏa mãn hệ ba tập ràng buộc nghiêm ngặt sau:

1. Ràng buộc hàng (Row Constraint):

$$\forall i \in [0, 8], \forall j, k \in [0, 8] \wedge j \neq k \implies x_{ij} \neq x_{ik}$$

2. Ràng buộc cột (Column Constraint):

$$\forall j \in [0, 8], \forall i, k \in [0, 8] \wedge i \neq k \implies x_{ij} \neq x_{kj}$$

3. Ràng buộc khối con 3×3 (Box Constraint):

$$\forall a, b \in [0, 2], \forall i, i' \in [3a, 3a+2], \forall j, j' \in [3b, 3b+2] \wedge (i, j) \neq (i', j') \implies x_{ij} \neq x_{i'j'}$$

1.1.2 Lý do Lựa chọn Thuật toán Quay lui (Backtracking)

Nhóm nghiên cứu quyết định lựa chọn thuật toán **Quay lui (Backtracking)** kết hợp kỹ thuật **Đệ quy (Recursion)** làm giải thuật cốt lõi dựa trên các luận cứ khoa học sau:

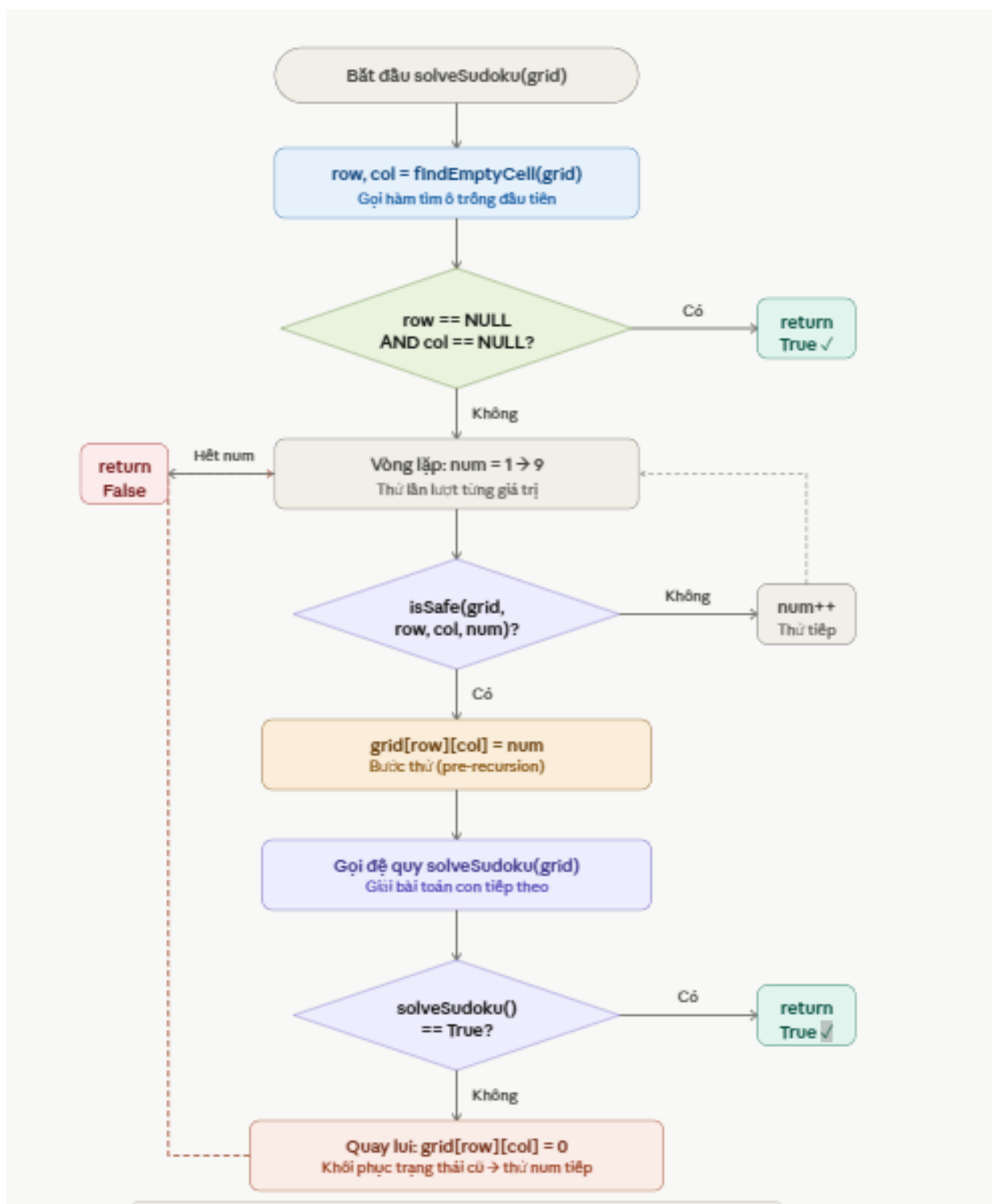
- **Tính toàn vẹn của không gian tìm kiếm:** Khác với các thuật toán tham lam (Greedy Algorithms) dễ bị rơi vào tối ưu cục bộ hoặc không tìm thấy lời giải đối với các cấu hình ma trận phức tạp, Quay lui đảm bảo duyệt một cách có hệ thống qua toàn bộ không gian cấu hình có thể, từ đó cam kết tìm ra lời giải chính xác nếu tồn tại, hoặc khẳng định bài toán vô nghiệm.

- **Cơ chế cắt nhánh sớm (Pruning) tối ưu:** Thuật toán áp dụng mô hình duyệt cây không gian trạng thái theo chiều sâu (*Depth-First Search* - *DFS*). Điểm vượt trội của Quay lui so với thuật toán duyệt cạn thông thường (Brute-Force) là khả năng kiểm tra điều kiện hợp lệ (*IsSafe*) ngay tại bước thử nghiệm giá trị. Nếu một nhánh giá trị vi phạm ràng buộc tại ô hiện tại, toàn bộ cây con tiềm năng phía dưới sẽ bị cắt bỏ ngay lập tức, giúp tiết kiệm đáng kể thời gian tính toán.
- **Tương thích hoàn hảo với cấu trúc dữ liệu:** Cấu trúc dữ liệu mảng hai chiều biểu diễn bàn cờ cho phép việc gán thử nghiệm giá trị (*Forward Step*) và khôi phục trạng thái cũ (*Backtracking Step*) diễn ra tức thời với độ phức tạp thời gian chỉ là $O(1)$.
- **Tính khả thi về mặt tài nguyên:** Với không gian cấu hình của Sudoku cấp độ 9×9 , cấu trúc bộ nhớ Stack phục vụ đệ quy của hệ thống có độ sâu tối đa chỉ là 81 tầng. Bộ nhớ tiêu tốn hoàn toàn nằm trong giới hạn vật lý an toàn của các kiến trúc máy tính hiện đại, không gây ra nguy cơ tràn bộ nhớ đệm (*Stack Overflow*).

1.2 Thiết kế và Triển khai Mã nguồn Mô phỏng thuật toán

1.2.1 Sơ đồ luồng thuật toán chi tiết (Flowchart)

Sơ đồ dưới đây được thiết kế phân tầng không gian tuyến tính nâng cao, đảm bảo các nhánh rẽ và đường hồi quy trạng thái đi vòng ngoài một cách độc lập, hoàn toàn không bị chồng chéo ký tự và hình khối trên hệ thống biên dịch:



Hình 1: Sơ đồ khối thuật toán

1.2.2 Biện giải chi tiết logic vận hành của Giải thuật Quay lui

Bản chất hoạt động của thuật toán Quay lui (*Backtracking*) tích hợp đệ quy trong bài toán Sudoku là quá trình xây dựng và thám hiểm một **Cây quyết định (Decision Tree)** theo chiến lược duyệt theo chiều sâu (*Depth-First Search - DFS*). Mỗi một ô trống trên lưới ma trận tương ứng với một tầng phân cấp của cây dữ liệu, và tập hợp các giá trị số nguyên từ 1 đến 9 biểu thị cho 9 nhánh rẽ tiềm năng. Tiến trình này được thực thi nghiêm ngặt thông qua chuỗi 5 bước khép kín dưới đây:

1. Bước 1 - Định vị không gian trạng thái (*FindEmptyCell*):

Thuật toán bắt đầu một tầng thực thi bằng việc quét tuần tự qua cấu trúc mảng hai chiều kích thước 9×9 từ trái sang phải và từ trên xuống dưới. Mục tiêu là xác định tọa độ hàng và cột (*row, col*) của ô đầu tiên chưa được gán dữ liệu (đang mang giá trị mặc định là 0).

Nếu ma trận không còn tồn tại bất kỳ ô trống nào, hệ thống đã đạt tới **Điều kiện dừng cơ sở (Base Case)** của tiến trình đệ quy. Lúc này, toàn bộ lưới ô số đã được thỏa mãn hệ ràng buộc, hàm lập tức kết thúc và phát tín hiệu **TRUE** truyền ngược về các tầng cha để nghiệm thu kết quả toàn cục.

2. Bước 2 - Thử nghiệm giá trị cục bộ (*Loop num 1-9*):

Tại vị trí ô trống vừa được thiết lập ở bước trước, một vòng lặp hữu hạn cục bộ kiểm thử từ số 1 đến số 9 được thiết lập. Vòng lặp này đóng vai trò sinh ra các hạt giống giả thuyết toán học. Chương trình sẽ nhét tuần tự từng con số lên nhằm phân tích tính khả thi, đảm bảo tính toàn vẹn và không bỏ sót bất kỳ phương án khả dĩ nào trong không gian tổ hợp.

3. Bước 3 - Kiểm tra điều kiện biên an toàn (*IsSafe*):

Trước khi chính thức cập nhật giá trị mới vào cấu trúc dữ liệu mảng tĩnh, con số thử nghiệm *num* bắt buộc phải vượt qua bộ lọc kiểm tra ràng buộc toán học thông qua hàm *IsSafe()*. Hàm kiểm tra đồng thời ba chiều thuộc tính: dòng ngang, cột dọc và khối vuông bao cục bộ 3×3 .

Nếu phát hiện sự trùng lặp (kết quả trả về **FALSE**), kỹ thuật **Cắt nhánh sớm (Pruning)** sẽ được kích hoạt nhằm bẻ gãy nhánh cây con vô nghiệm hiện tại, bỏ qua số *num* này và lập tức dịch chuyển sang chu kỳ lặp tiếp theo. Nếu kết quả trả về **TRUE**, giá trị số được tạm ghi nhận vào bộ nhớ: `grid[row][col] = num`.

4. Bước 4 - Thám hiểm sâu bằng cơ chế đệ quy (Recursive Call):

Ngay sau khi xác lập một cấu hình tạm thời hợp lệ tại ô hiện tại, thuật toán tiến hành lệnh gọi đệ quy lồng nhau: `SolveSudoku(grid)`.

Về mặt vật lý máy tính, khung sườn ngữ cảnh của hàm hiện tại (bao gồm các con trỏ lệnh, biến cục bộ tầng dòng/cột) sẽ được đóng gói gọn gàng và đẩy vào ngăn xếp hệ thống **Call Stack**. Hệ điều hành sẽ tạm khóa tầng xử lý hiện tại và đưa CPU lặn sâu xuống tầng kế tiếp của cây quyết định nhằm lặp lại chu trình tìm kiếm ô trống cho vị trí tiếp theo.

5. Bước 5 - Hồi quy và Quay lui phục hồi trạng thái (Backtracking Step):

Đây là bước tư duy mấu chốt cấu thành nên bản chất giải thuật. Nếu ở các tầng đệ quy sâu hơn phía dưới, chương trình rơi vào trạng thái bế tắc (quét gặp một ô trống mà thử nghiệm mọi số từ 1 đến 9 đều vi phạm quy tắc an toàn của Sudoku), điều đó đồng nghĩa với việc các bước giả định ở tầng trên đã chọn sai hướng đi.

Khi nhận tín hiệu thất bại **FALSE** từ tầng dưới truyền hồi quy lên, hàm hiện tại thực hiện tiến trình **Quay lui**: Xóa bỏ cấu hình lỗi bằng cách gán hoàn nguyên trạng thái `grid[row][col] = 0`, giải phóng vùng đệm Stack, rồi tăng giá trị số chạy lên $num + 1$ để tái thiết lập lại quy trình thử nghiệm tại Bước 3.

Để phục vụ cho công tác cài đặt mã nguồn chính xác, giải thuật được đặc tả chi tiết dưới dạng mã giả cấu trúc toán học cụ thể đối với từng module hàm thành phần.

1.2.3 Hàm Định vị Ô trống trên Ma trận (`findEmptyCell`)

Hàm thực hiện quét tuần tự ma trận từ trên xuống dưới, từ trái sang phải nhằm tìm kiếm tọa độ của ô đầu tiên chưa được điền số (được quy ước lưu trữ giá trị 0).

Input: mảng hai chiều `grid[9][9]`

Output: Tọa độ (row, col) của ô trống đầu tiên tìm thấy, hoặc (NULL, NULL) nếu bảng đầy

Listing 1: Giả mã hàm tìm kiếm ô trống

```
1 int findEmptyCell(int grid[][9]):  
2     //Duyệt qua từng hàng từ 0 đến 8  
3     for row = 0 to row = 8 do  
4     //Duyệt qua từng cột từ 0 đến 8
```

```

5         for col = 0 to col = 8 do
6             //Neu tim thay gia tri 0 (o trong)
7             if grid[row][col] == 0 then
8                 return row, col
9
10        //Neu duyet het bang ma khong thay o nao bang 0
11        return NULL, NULL

```

1.2.4 Hàm Kiểm tra Ràng buộc Toán học (IsSafe)

Module chịu trách nhiệm thẩm định tính hợp lệ của một cấu hình khi dự kiến điền giá trị `num` vào vị trí tọa độ (`row`, `col`). Hàm cô lập và kiểm tra tuân tự 3 luật ràng buộc của trò chơi.

Input: Ma trận `grid[][]`, vị trí hàng `row`, vị trí cột `col`, giá trị `num` cần thử nghiệm.

Output: TRUE nếu điền được, FALSE nếu vi phạm bất kỳ ràng buộc nào.

Listing 2: Giả mã hàm kiểm tra ràng buộc hợp lệ

```

1 Bool isSafe(int grid[][], int row, int col, int num):
2     // 1. Kiem tra hang (Row Constraint)
3     for j = 0 to j = 8 do
4         if grid[row][j] == num then
5             return False
6
7     // 2. Kiem tra cot (Column Constraint)
8     for i = 0 to i = 8 do
9         if grid[i][col] == num then
10            return False
11
12    // 3. Kiem tra vung 3x3 (Box Constraint)
13    // Xac dinh goc tren ben trai cua vung 3x3 chua o(row, col)
14    startRow = row - (row mod 3)
15    startCol = col - (col mod 3)
16
17    for i = 0 to i = 2 do
18        for j = 0 to j = 2 do
19            if grid[startRow + i][startCol + j] == num then
20                return False
21
22    // Neu khogn vi pham bat ky dieu kien nao
23    return True

```

1.2.5 Hàm Thực thi Đệ quy Quay lui Chủ đạo (SolveSudoku)

Đây là trái tim của hệ thống phần mềm, tích hợp cơ chế điều khiển đệ quy để triển khai tìm kiếm chuyên sâu trên cây trạng thái.

Input: Ma trận `grid[][]` tại trạng thái hiện tại.

Output: TRUE nếu tìm được lời giải toàn vẹn, FALSE nếu trạng thái nhánh hiện tại vô nghiệm.

Listing 3: Giải mã giải thuật đệ quy quay lui chủ đạo

```
1 bool solveSudoku(int grid[] []):
2     // 1. Tìm vị trí ô trống (row, col)
3     row, col = findEmptyCell(grid)
4
5     // 2. Base Case (Điều kiện dừng)
6     // Nếu không còn ô trống nào, tất cả các ô được điền hợp lệ
7     if row == NULL and col == NULL then
8         return True
9
10    // 3. Recursive Step (Bước đệ quy)
11    // Thu các giá trị khả thi từ 1 đến 9 cho ô trống vừa tìm được
12    for num = 1 to num = 9 do
13        // Kiểm tra điều kiện ràng buộc (Constraint Satisfaction)
14        if isSafe(grid, row, col, num) == True then
15            // Bước thu (Pre-recursion): Gán giá trị vào CTDL
16            grid[row][col] = num
17
18            // Gọi đệ quy để giải bài toán con tiếp theo
19            if solveSudoku(grid) == True then
20                return True
21
22            // Bước QUAY LUI (Backtrack): Khôi phục trạng thái cũ
23            của grid
24            // Nếu việc thu 'num' dẫn đến ngõ cụt
25            grid[row][col] = 0
26
27    // 4. Trả về False nếu không có giá trị nào từ 1-9 thỏa mãn
28    (Dead end)
29    return False
```

1.3 Phân tích Chuyên sâu Độ phức tạp của Giải thuật

Để đánh giá hiệu năng và mức độ chiếm dụng tài nguyên của thuật toán quay lui đệ quy đối với bài toán Sudoku 9×9 , ta tiến hành phân tích toán học dựa trên ký pháp Đại O lớn (*Big O Notation*).

1.3.1 Độ phức tạp Thời gian (Time Complexity)

- **Trường hợp xấu nhất (Worst-case Scenario):**

Xảy ra khi gặp một ma trận Sudoku trống hoàn toàn hoặc có cấu trúc phân bố các ô mỗi được thiết kế đặc biệt nhằm bẫy thuật toán, buộc chương trình phải duyệt qua gần như toàn bộ không gian trạng thái. Gọi m là số lượng ô trống cần phải điền trên bảng ($0 \leq m \leq 81$). Tại mỗi ô trống, thuật toán có tối đa 9 khả năng lựa chọn (từ 1 đến 9). Do đó, cây quyết định đệ quy sẽ có hệ số nhánh tối đa là 9 và độ sâu cực đại của cây là m .

Số lượng nút lá tối đa mà thuật toán cần duyệt qua tiến tới giá trị chặn trên là 9^m . Tại mỗi nút, để thực hiện phép kiểm tra cắt nhánh `IsSafe`, thuật toán cần thực hiện:

$$\text{Số phép toán} = 9 \text{ (hàng)} + 9 \text{ (cột)} + 9 \text{ (khối)} = 27 \text{ phép tính} \implies O(1)$$

Do kích thước bảng $N = 9$ cố định, chi phí kiểm tra tại mỗi bước là một hằng số $O(1)$. Do đó, tổng độ phức tạp thời gian trong trường hợp xấu nhất được biểu diễn bằng biểu thức toán học:

$$T(n) = O(9^m)$$

Với một ma trận trống hoàn toàn ($m = 81$), độ phức tạp lý thuyết là $O(9^{81})$. Tuy nhiên, trong thực tế, các ràng buộc sẵn có trên bàn cờ làm cho hệ số nhánh thực tế giảm mạnh xuống rất nhỏ hơn 9, giúp chương trình chạy xong chỉ trong vài mili-giây.

- **Trường hợp tốt nhất (Best-case Scenario):**

Xảy ra khi các ô mỗi có sẵn trên bảng chỉ dẫn hướng đi chính xác duy nhất. Thuật toán chọn số nào là đúng số đó tại mọi ô trống mà không bao giờ kích hoạt bước quay lui về giá trị 0. Khi đó, số lần gọi hàm tỷ lệ tuyến tính hoàn toàn với số ô trống m :

$$T(n) = O(m)$$

1.3.2 Độ phức tạp Không gian bộ nhớ (Space Complexity)

Độ phức tạp không gian của chương trình bị chi phối bởi hai thành phần cấu trúc bộ nhớ chính:

1. **Bộ nhớ lưu trữ cấu trúc dữ liệu tĩnh:** Ma trận Sudoku được lưu bằng mảng 2 chiều kích thước cố định 9×9 . Không gian bộ nhớ này là bất biến và tồn:

$$S_{\text{data}} = 9 \times 9 \times \text{sizeof}(\text{int}) \implies O(1)$$

2. **Bộ nhớ động Hệ thống (Call Stack Framework):** Do sử dụng giải thuật đệ quy theo chiều sâu (DFS), hệ thống bắt buộc phải duy trì một Stack đệ quy để lưu trữ các biến cục bộ, con trỏ lệnh và tham số hàm của các tầng đệ quy chưa kết thúc. Độ sâu cực đại của Stack này tương ứng với số lượng ô trống tối đa m trên ma trận:

$$S_{\text{stack}} = O(m)$$

Vì $m \leq 81$, không gian Stack cần thiết tối đa là $O(81)$, tương đương một lượng bộ nhớ rất nhỏ chỉ vài Kilobytes. Tổng độ phức tạp không gian lưu trữ tổng thể của giải thuật đạt mức tối ưu cao:

$$S(n) = S_{\text{data}} + S_{\text{stack}} = O(1) + O(m) = O(m)$$